Chapter 10. *Avoiding Liveness Hazards*

There is often a tension between safety and liveness. We use locking to ensure thread safety, but indiscriminate use of locking can cause *lock-ordering deadlocks*. Similarly, we use thread pools and semaphores to bound resource consumption, but failure to understand the activities being bounded can cause *resource deadlocks*. Java applications do not recover from deadlock, so it is worthwhile to ensure that your design precludes the conditions that could cause it. This chapter explores some of the causes of liveness failures and what can be done to prevent them.

10.1. Deadlock

Deadlock is illustrated by the classic, if somewhat unsanitary, “dining philosophers” problem. Five philosophers go out for Chinese food and are seated at a circular table. There are five chopsticks (not five pairs), one placed between each pair of diners. The philosophers alternate between thinking and eating. Each needs to acquire two chopsticks for long enough to eat, but can then put the chopsticks back and return to thinking. There are some chopstick-management algorithms that let everyone eat on a more or less timely basis (a hungry philosopher tries to grab both adjacent chopsticks, but if one is not available, puts down the one that is available and waits a minute or so before trying again), and some that can result in some or all of the philosophers dying of hunger (each philosopher immediately grabs the chopstick to his left and waits for the chopstick to his right to be available before putting down the left). The latter situation, where each has a resource needed by another and is waiting for a resource held by another, and will not release the one they hold until they acquire the one they don't, illustrates deadlock.

When a thread holds a lock forever, other threads attempting to acquire that lock will block forever waiting. When thread *A* holds lock *L* and tries to acquire lock *M*, but at the same time thread *B* holds *M* and tries to ac-

quire *L*, *both* threads will wait forever. This situation is the simplest case of deadlock (or *deadly embrace*), where multiple threads wait forever due to a cyclic locking dependency. (Think of the threads as the nodes of a directed graph whose edges represent the relation "Thread *A* is waiting for a resource held by thread *B*". If this graph is cyclical, there is a deadlock.)

Database systems are designed to detect and recover from deadlock. A transaction may acquire many locks, and locks are held until the transaction commits. So it is quite possible, and in fact not uncommon, for two transactions to deadlock. Without intervention, they would wait forever (holding locks that are probably required by other transactions as well). But the database server is not going to let this happen. When it detects that a set of transactions is deadlocked (which it does by searching the *is-waiting-for* graph for cycles), it picks a victim and aborts that transaction. This releases the locks held by the victim, allowing the other transactions to proceed. The application can then retry the aborted transaction, which may be able to complete now that any competing transactions have completed.

The JVM is not nearly as helpful in resolving deadlocks as database servers are. When a set of Java threads deadlock, that's the end of the game—those threads are permanently out of commission. Depending on what those threads do, the application may stall completely, or a particular subsystem may stall, or performance may suffer. The only way to restore the application to health is to abort and restart it—and hope the same thing doesn't happen again.

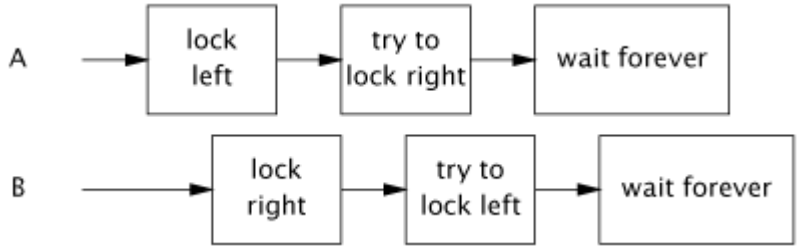
Like many other concurrency hazards, deadlocks rarely manifest themselves immediately. The fact that a class has a potential deadlock doesn't mean that it ever *will* deadlock, just that it can. When deadlocks do manifest themselves, it is often at the worst possible time—under heavy production load.

10.1.1. Lock-ordering Deadlocks

`LeftRightDeadlock` in **Listing 10.1** is at risk for deadlock. The `leftRight` and `rightLeft` methods each acquire the `left` and `right`

locks. If one thread calls `leftRight` and another calls `rightLeft`, and their actions are interleaved as shown in **Figure 10.1**, they will deadlock.

Figure 10.1. Unlucky Timing in `LeftRightDeadlock`.



The deadlock in `LeftRightDeadlock` came about because the two threads attempted to acquire the same locks in a *different order*. If they asked for the locks in the same order, there would be no cyclic locking dependency and therefore no deadlock. If you can guarantee that every thread that needs locks *L* and *M* at the same time always acquires *L* and *M* in the same order, there will be no deadlock.

A program will be free of lock-ordering deadlocks if all threads acquire the locks they need in a fixed global order.

Verifying consistent lock ordering requires a global analysis of your program's locking behavior. It is not sufficient to inspect code paths that acquire multiple locks individually; both `leftRight` and `rightLeft` are "reasonable" ways to acquire the two locks, they are just not compatible. When it comes to locking, the left hand needs to know what the right hand is doing.

Listing 10.1. Simple Lock-ordering Deadlock. *Don't do this.*

```
// Warning: deadlock-prone!
public class LeftRightDeadlock {
    private final Object left = new Object();
    private final Object right = new Object();

    public void leftRight() {
        synchronized (left) {
            synchronized (right) {
                doSomething();
            }
        }
    }

    public void rightLeft() {
        synchronized (right) {
            synchronized (left) {
                doSomethingElse();
            }
        }
    }
}
```

10.1.2. Dynamic Lock Order Deadlocks

Sometimes it is not obvious that you have sufficient control over lock ordering to prevent deadlocks. Consider the harmless-looking code in **Listing 10.2** that transfers funds from one account to another. It acquires the locks on both `Account` objects before executing the transfer, ensuring that the balances are updated atomically and without violating invariants such as "an account cannot have a negative balance".

How can `transferMoney` deadlock? It may appear as if all the threads acquire their locks in the same order, but in fact the lock order depends on the order of arguments passed to `transferMoney`, and these in turn might depend on external inputs. Deadlock can occur if two threads call `transferMoney` at the same time, one transferring from *X* to *Y*, and the other doing the opposite:

Listing 10.2. Dynamic Lock-ordering Deadlock. *Don't do this.*

```
// Warning: deadlock-prone!
public void transferMoney(Account fromAccount,
                          Account toAccount,
                          DollarAmount amount)
        throws InsufficientFundsException {
    synchronized (fromAccount) {
        synchronized (toAccount) {
            if (fromAccount.getBalance().compareTo(amount) < 0)
                throw new InsufficientFundsException();
            else {
                fromAccount.debit(amount);
                toAccount.credit(amount);
            }
        }
    }
}
```

```
A: transferMoney(myAccount, yourAccount, 10);
B: transferMoney(yourAccount, myAccount, 20);
```

With unlucky timing, *A* will acquire the lock on `myAccount` and wait for the lock on `yourAccount`, while *B* is holding the lock on `yourAccount` and waiting for the lock on `myAccount`.

Deadlocks like this one can be spotted the same way as in **Listing 10.1**—look for nested lock acquisitions. Since the order of arguments is out of our control, to fix the problem we must *induce* an ordering on the locks and acquire them according to the induced ordering consistently throughout the application.

One way to induce an ordering on objects is to use `System.identityHashCode`, which returns the value that would be returned by `Object.hashCode`. **Listing 10.3** shows a version of `transferMoney` that uses `System.identityHashCode` to induce a lock ordering. It involves a few extra lines of code, but eliminates the possibility of deadlock.

In the rare case that two objects have the same hash code, we must use an arbitrary means of ordering the lock acquisitions, and this reintroduces the possibility of deadlock. To prevent inconsistent lock ordering in this case, a third "tie breaking" lock is used. By acquiring the tie-breaking lock before acquiring either `Account` lock, we ensure that only one thread at a time performs the risky task of acquiring two locks in an arbitrary or-

der, eliminating the possibility of deadlock (so long as this mechanism is used consistently). If hash collisions were common, this technique might become a concurrency bottleneck (just as having a single, program-wide lock would), but because hash collisions with `System.identityHashCode` are vanishingly infrequent, this technique provides that last bit of safety at little cost.

Listing 10.3. Inducing a Lock Ordering to Avoid Deadlock.

```
private static final Object tieLock = new Object();

public void transferMoney(final Account fromAcct,
                          final Account toAcct,
                          final DollarAmount amount)
        throws InsufficientFundsException {
    class Helper {
        public void transfer() throws InsufficientFundsException {
            if (fromAcct.getBalance().compareTo(amount) < 0)
                throw new InsufficientFundsException();
            else {
                fromAcct.debit(amount);
                toAcct.credit(amount);
            }
        }
    }
    int fromHash = System.identityHashCode(fromAcct);
    int toHash = System.identityHashCode(toAcct);

    if (fromHash < toHash) {
        synchronized (fromAcct) {
            synchronized (toAcct) {
                new Helper().transfer();
            }
        }
    } else if (fromHash > toHash) {
        synchronized (toAcct) {
            synchronized (fromAcct) {
                new Helper().transfer();
            }
        }
    } else {
        synchronized (tieLock) {
            synchronized (fromAcct) {
                synchronized (toAcct) {
                    new Helper().transfer();
                }
            }
        }
    }
}
```

If `Account` has a unique, immutable, comparable key such as an account number, inducing a lock ordering is even easier: order objects by their key, thus eliminating the need for the tie-breaking lock.

You may think we're overstating the risk of deadlock because locks are usually held only briefly, but deadlocks are a serious problem in real systems. A production application may perform billions of lock acquire-release cycles per day. Only one of those needs to be timed just wrong to bring the application to deadlock, and even a thorough load-testing regimen may not disclose all latent deadlocks.[1] `DemonstrateDeadlock` in Listing 10.4[2] deadlocks fairly quickly on most systems.

[1] Ironically, holding locks for short periods of time, as you are supposed to do to reduce lock contention, increases the likelihood that testing will not disclose latent deadlock risks.

[2] For simplicity, `DemonstrateDeadlock` ignores the issue of negative account balances.

Listing 10.4. Driver Loop that Induces Deadlock Under Typical Conditions.

```
public class DemonstrateDeadlock {
    private static final int NUM_THREADS = 20;
    private static final int NUM_ACCOUNTS = 5;
    private static final int NUM_ITERATIONS = 1000000;

    public static void main(String[] args) {
        final Random rnd = new Random();
        final Account[] accounts = new Account[NUM_ACCOUNTS];

        for (int i = 0; i < accounts.length; i++)
            accounts[i] = new Account();

        class TransferThread extends Thread {
            public void run() {
                for (int i=0; i<NUM_ITERATIONS; i++) {
                    int fromAcct = rnd.nextInt(NUM_ACCOUNTS);
                    int toAcct = rnd.nextInt(NUM_ACCOUNTS);
                    DollarAmount amount =
                        new DollarAmount(rnd.nextInt(1000));
                    transferMoney(accounts[fromAcct],
                                  accounts[toAcct], amount);
                }
            }
        }
        for (int i = 0; i < NUM_THREADS; i++)
            new TransferThread().start();
    }
}
```

10.1.3. Deadlocks Between Cooperating Objects

Multiple lock acquisition is not always as obvious as in `LeftRightDeadlock` or `transferMoney`; the two locks need not be acquired by the same method. Consider the cooperating classes in **Listing 10.5**, which might be used in a taxicab dispatching application. `Taxi` represents an individual taxi with a location and a destination; `Dispatcher` represents a fleet of taxis.

While no method *explicitly* acquires two locks, callers of `setLocation` and `getImage` can acquire two locks just the same. If a thread calls `setLocation` in response to an update from a GPS receiver, it first updates the taxi's location and then checks to see if it has reached its destination. If it has, it informs the dispatcher that it needs a new destination. Since both `setLocation` and `notifyAvailable` are `synchronized`, the thread calling `setLocation` acquires the `Taxi` lock and then the `Dispatcher` lock. Similarly, a thread calling `getImage` acquires the `Dispatcher` lock and then each `Taxi` lock (one at at time). Just as in `LeftRightDeadlock`, two locks are acquired by two threads in different orders, risking deadlock.

It was easy to spot the deadlock possibility in `LeftRightDeadlock` or `transferMoney` by looking for methods that acquire two locks. Spotting the deadlock possibility in `Taxi` and `Dispatcher` is a little harder: the warning sign is that an *alien* method (defined on page 40) is being called while holding a lock.

Invoking an alien method with a lock held is asking for liveness trouble. The alien method might acquire other locks (risking deadlock) or block for an unexpectedly long time, stalling other threads that need the lock you hold.

10.1.4. Open Calls

Of course, `Taxi` and `Dispatcher` didn't *know* that they were each half of a deadlock waiting to happen. And they shouldn't have to; a method call is an abstraction barrier intended to shield you from the details of

what happens on the other side. But because you don't know what is happening on the other side of the call, *calling an alien method with a lock held is difficult to analyze and therefore risky.*

Calling a method with no locks held is called an *open call* [CPJ 2.4.1.3], and classes that rely on open calls are more well-behaved and composable than classes that make calls with locks held. Using open calls to avoid deadlock is analogous to using encapsulation to provide thread safety: while one can certainly construct a thread-safe program without any encapsulation, the thread safety analysis of a program that makes effective use of encapsulation is far easier than that of one that does not. Similarly, the liveness analysis of a program that relies exclusively on open calls is far easier than that of one that does not. Restricting yourself to open calls makes it far easier to identify the code paths that acquire multiple locks and therefore to ensure that locks are acquired in a consistent order.[3]

[3] The need to rely on open calls and careful lock ordering reflects the fundamental messiness of composing synchronized objects rather than synchronizing composed objects.

Listing 10.5. Lock-ordering Deadlock Between Cooperating Objects. *Don't do this.*

```
// Warning: deadlock-prone!
class Taxi {
    @GuardedBy("this") private Point location, destination;
    private final Dispatcher dispatcher;

    public Taxi(Dispatcher dispatcher) {
        this.dispatcher = dispatcher;
    }

    public synchronized Point getLocation() {
        return location;
    }

    public synchronized void setLocation(Point location) {
        this.location = location;
        if (location.equals(destination))
            dispatcher.notifyAvailable(this);
    }
}

class Dispatcher {
    @GuardedBy("this") private final Set<Taxi> taxis;
    @GuardedBy("this") private final Set<Taxi> availableTaxis;

    public Dispatcher() {
        taxis = new HashSet<Taxi>();
        availableTaxis = new HashSet<Taxi>();
    }

    public synchronized void notifyAvailable(Taxi taxi) {
        availableTaxis.add(taxi);
    }

    public synchronized Image getImage() {
        Image image = new Image();
        for (Taxi t : taxis)
            image.drawMarker(t.getLocation());
        return image;
    }
}
```

`Taxi` and `Dispatcher` in **Listing 10.5** can be easily refactored to use open calls and thus eliminate the deadlock risk. This involves shrinking the `synchronized` blocks to guard only operations that involve shared state, as in **Listing 10.6**. Very often, the cause of problems like those in **Listing 10.5** is the use of `synchronized` methods instead of smaller `synchronized` blocks for reasons of compact syntax or simplicity rather than because the entire method must be guarded by a lock. (As a bonus, shrinking the `synchronized` block may also improve scalability as well; see **Section 11.4.1** for advice on sizing `synchronized` blocks.)

Strive to use open calls throughout your program. Programs that rely on open calls are far easier to analyze for deadlock-freedom than those that allow calls to alien methods with locks held.

Restructuring a `synchronized` block to allow open calls can sometimes have undesirable consequences, since it takes an operation that was atomic and makes it not atomic. In many cases, the loss of atomicity is perfectly acceptable; there's no reason that updating a taxi's location and notifying the dispatcher that it is ready for a new destination need be an atomic operation. In other cases, the loss of atomicity is noticeable but the semantic changes are still acceptable. In the deadlock-prone version, `getImage` produces a complete snapshot of the fleet locations at that instant; in the refactored version, it fetches the location of each taxi at slightly different times.

In some cases, however, the loss of atomicity is a problem, and here you will have to use another technique to achieve atomicity. One such technique is to structure a concurrent object so that only one thread can execute the code path following the open call. For example, when shutting down a service, you may want to wait for in-progress operations to complete and then release resources used by the service. Holding the service lock while waiting for operations to complete is inherently deadlock-prone, but releasing the service lock before the service is shut down may let other threads start new operations. The solution is to hold the lock long enough to update the service state to “shutting down” so that other threads wanting to start new operations—including shutting down the service—see that the service is unavailable, and do not try. You can then wait for shutdown to complete, knowing that only the shutdown thread has access to the service state after the open call completes. Thus, rather than using locking to keep the other threads out of a critical section of code, this technique relies on constructing protocols so that other threads don't try to get in.

10.1.5. Resource Deadlocks

Just as threads can deadlock when they are each waiting for a lock that the other holds and will not release, they can also deadlock when waiting for resources.

Listing 10.6. Using open calls to avoiding deadlock between cooperating objects.

```
@ThreadSafe
class Taxi {
    @GuardedBy("this") private Point location, destination;
    private final Dispatcher dispatcher;
    ...
    public synchronized Point getLocation() {
        return location;
    }

    public void setLocation(Point location) {
        boolean reachedDestination;
        synchronized (this) {
            this.location = location;
            reachedDestination = location.equals(destination);
        }
        if (reachedDestination)
            dispatcher.notifyAvailable(this);
    }
}

@ThreadSafe
class Dispatcher {
    @GuardedBy("this") private final Set<Taxi> taxis;
    @GuardedBy("this") private final Set<Taxi> availableTaxis;
    ...
    public synchronized void notifyAvailable(Taxi taxi) {
        availableTaxis.add(taxi);
    }

    public Image getImage() {
        Set<Taxi> copy;
        synchronized (this) {
            copy = new HashSet<Taxi>(taxis);
        }
        Image image = new Image();
        for (Taxi t : copy)
            image.drawMarker(t.getLocation());
        return image;
    }
}
```

Say you have two pooled resources, such as connection pools for two different databases. Resource pools are usually implemented with semaphores (see **Section 5.5.3**) to facilitate blocking when the pool is empty. If a task requires connections to both databases and the two resources are not always requested in the same order, thread A could be holding a connection to database D_1 while waiting for a connection to database D_2, and thread B could be holding a connection to D_2 while waiting for a connec-

tion to D_1. (The larger the pools are, the less likely this is to occur; if each pool has N connections, deadlock requires N sets of cyclically waiting threads and a lot of unlucky timing.)

Another form of resource-based deadlock is *thread-starvation deadlock*. We saw an example of this hazard in **Section 8.1.1**, where a task that submits a task and waits for its result executes in a single-threaded `Executor`. In that case, the first task will wait forever, permanently stalling that task and all others waiting to execute in that `Executor`. Tasks that wait for the results of other tasks are the primary source of thread-starvation deadlock; bounded pools and interdependent tasks do not mix well.

10.2. Avoiding and Diagnosing Deadlocks

A program that never acquires more than one lock at a time cannot experience lock-ordering deadlock. Of course, this is not always practical, but if you can get away with it, it's a lot less work. If you must acquire multiple locks, lock ordering must be a part of your design: try to minimize the number of potential locking interactions, and follow and document a lock-ordering protocol for locks that may be acquired together.

In programs that use fine-grained locking, audit your code for deadlock freedom using a two-part strategy: first, identify where multiple locks could be acquired (try to make this a small set), and then perform a global analysis of all such instances to ensure that lock ordering is consistent across your entire program. Using open calls wherever possible simplifies this analysis substantially. With no non-open calls, finding instances where multiple locks are acquired is fairly easy, either by code review or by automated bytecode or source code analysis.

10.2.1. Timed Lock Attempts

Another technique for detecting and recovering from deadlocks is to use the timed `tryLock` feature of the explicit `Lock` classes (see **Chapter 13**) instead of intrinsic locking. Where intrinsic locks wait forever if they cannot acquire the lock, explicit locks let you specify a timeout after which `tryLock` returns failure. By using a timeout that is much longer than

you expect acquiring the lock to take, you can regain control when something unexpected happens. (**Listing 13.3** on page 280 shows an alternative implementation of `transferMoney` using the polled `tryLock` with retries for probabilistic deadlock avoidance.)

When a timed lock attempt fails, you do not necessarily know *why*. Maybe there was a deadlock; maybe a thread erroneously entered an infinite loop while holding that lock; or maybe some activity is just running a lot slower than you expected. Still, at least you have the opportunity to record that your attempt failed, log any useful information about what you were trying to do, and restart the computation somewhat more gracefully than killing the entire process.

Using timed lock acquisition to acquire multiple locks can be effective against deadlock even when timed locking is not used consistently throughout the program. If a lock acquisition times out, you can release the locks, back off and wait for a while, and try again, possibly clearing the deadlock condition and allowing the program to recover. (This technique works only when the two locks are acquired together; if multiple locks are acquired due to the nesting of method calls, you cannot just release the outer lock, even if you know you hold it.)

10.2.2. Deadlock Analysis with Thread Dumps

While preventing deadlocks is mostly your problem, the JVM can help identify them when they do happen using *thread dumps*. A thread dump includes a stack trace for each running thread, similar to the stack trace that accompanies an exception. Thread dumps also include locking information, such as which locks are held by each thread, in which stack frame they were acquired, and which lock a blocked thread is waiting to acquire.[4] Before generating a thread dump, the JVM searches the is-waiting-for graph for cycles to find deadlocks. If it finds one, it includes deadlock information identifying which locks and threads are involved, and where in the program the offending lock acquisitions are.

[4] This information is useful for debugging even when you don't have a deadlock; periodically triggering thread dumps lets you observe your program's locking behavior.

To trigger a thread dump, you can send the JVM process a `SIGQUIT` signal (`kill -3`) on Unix platforms, or press the `Ctrl-\` key on Unix or `Ctrl-Break` on Windows platforms. Many IDEs can request a thread dump as well.

If you are using the explicit `Lock` classes instead of intrinsic locking, Java 5.0 has no support for associating `Lock` information with the thread dump; explicit `Lock` s do not show up at all in thread dumps. Java 6 does include thread dump support and deadlock detection with explicit `Lock` s, but the information on where `Lock` s are acquired is necessarily less precise than for intrinsic locks. Intrinsic locks are associated with the stack frame in which they were acquired; explicit `Lock` s are associated only with the acquiring thread.

Listing 10.7 shows portions of a thread dump taken from a production J2EE application. The failure that caused the deadlock involves three components—a J2EE application, a J2EE container, and a JDBC driver, each from different vendors. (The names have been changed to protect the guilty.) All three were commercial products that had been through extensive testing cycles; each had a bug that was harmless until they all interacted and caused a fatal server failure.

We've shown only the portion of the thread dump relevant to identifying the deadlock. The JVM has done a lot of work for us in diagnosing the deadlock—which locks are causing the problem, which threads are involved, which other locks they hold, and whether other threads are being indirectly inconvenienced. One thread holds the lock on the `MumbleDBConnection` and is waiting to acquire the lock on the `MumbleDBCallableStatement` ; the other holds the lock on the `MumbleDBCallableStatement` and is waiting for the lock on the `MumbleDBConnection` .

Listing 10.7. Portion of Thread Dump After Deadlock.

```
Found one Java-level deadlock:
============================
"ApplicationServerThread":
  waiting to lock monitor 0x080f0cdc (a MumbleDBConnection),
  which is held by "ApplicationServerThread"
"ApplicationServerThread":
  waiting to lock monitor 0x080f0ed4 (a MumbleDBCallableStatement),
  which is held by "ApplicationServerThread"

Java stack information for the threads listed above:
"ApplicationServerThread":
        at MumbleDBConnection.remove_statement
        - waiting to lock <0x650f7f30> (a MumbleDBConnection)
        at MumbleDBStatement.close
        - locked <0x6024ffb0> (a MumbleDBCallableStatement)
     ...

"ApplicationServerThread":
        at MumbleDBCallableStatement.sendBatch
        - waiting to lock <0x6024ffb0> (a MumbleDBCallableStatement)
        at MumbleDBConnection.commit
        - locked <0x650f7f30> (a MumbleDBConnection)
     ...
```

The JDBC driver being used here clearly has a lock-ordering bug: different call chains through the JDBC driver acquire multiple locks in different orders. But this problem would not have manifested itself were it not for another bug: multiple threads were trying to use the same JDBC `Connection` at the same time. This was not how the application was supposed to work—the developers were surprised to see the same `Connection` used concurrently by two threads. There's nothing in the JDBC specification that requires a `Connection` to be thread-safe, and it is common to confine use of a `Connection` to a single thread, as was intended here. This vendor tried to deliver a thread-safe JDBC driver, as evidenced by the synchronization on multiple JDBC objects within the driver code. Unfortunately, because the vendor did not take lock ordering into account, the driver was prone to deadlock, but it was only the interaction of the deadlock-prone driver and the incorrect `Connection` sharing by the application that disclosed the problem. Because neither bug was fatal in isolation, both persisted despite extensive testing.

10.3. Other Liveness Hazards

While deadlock is the most widely encountered liveness hazard, there are several other liveness hazards you may encounter in concurrent pro-

grams including starvation, missed signals, and livelock. (Missed signals are covered in **Section 14.2.3**.)

10.3.1. Starvation

Starvation occurs when a thread is perpetually denied access to resources it needs in order to make progress; the most commonly starved resource is CPU cycles. Starvation in Java applications can be caused by inappropriate use of thread priorities. It can also be caused by executing nonterminating constructs (infinite loops or resource waits that do not terminate) with a lock held, since other threads that need that lock will never be able to acquire it.

The thread priorities defined in the Thread API are merely scheduling hints. The Thread API defines ten priority levels that the JVM can map to operating system scheduling priorities as it sees fit. This mapping is platform-specific, so two Java priorities can map to the same OS priority on one system and different OS priorities on another. Some operating systems have fewer than ten priority levels, in which case multiple Java priorities map to the same OS priority.

Operating system schedulers go to great lengths to provide scheduling fairness and liveness beyond that required by the Java Language Specification. In most Java applications, all application threads have the same priority, `Thread. NORM_PRIORITY` . The thread priority mechanism is a blunt instrument, and it's not always obvious what effect changing priorities will have; boosting a thread's priority might do nothing or might always cause one thread to be scheduled in preference to the other, causing starvation.

It is generally wise to resist the temptation to tweak thread priorities. As soon as you start modifying priorities, the behavior of your application becomes platform-specific and you introduce the risk of starvation. You can often spot a program that is trying to recover from priority tweaking or other responsiveness problems by the presence of `Thread.sleep` or `Thread.yield` calls in odd places, in an attempt to give more time to lower-priority threads.[5]

[5] The semantics of `Thread.yield` (and `Thread.sleep(0)`) are undefined [JLS 17.9]; the JVM is free to implement them as no-ops or treat them as scheduling hints. In particular, they are *not* required to have the semantics of `sleep(0)` on Unix systems—put the current thread at the end of the run queue for that priority, yielding to other threads of the same priority—though some JVMs implement `yield` in this way.

Avoid the temptation to use thread priorities, since they increase platform dependence and can cause liveness problems. Most concurrent applications can use the default priority for all threads.

10.3.2. Poor Responsiveness

One step removed from starvation is poor responsiveness, which is not uncommon in GUI applications using background threads. **Chapter 9** developed a framework for offloading long-running tasks onto background threads so as not to freeze the user interface. CPU-intensive background tasks can still affect responsiveness because they can compete for CPU cycles with the event thread. This is one case where altering thread priorities makes sense; when computeintensive background computations would affect responsiveness. If the work done by other threads are truly background tasks, lowering their priority can make the foreground tasks more responsive.

Poor responsiveness can also be caused by poor lock management. If a thread holds a lock for a long time (perhaps while iterating a large collection and performing substantial work for each element), other threads that need to access that collection may have to wait a very long time.

10.3.3. Livelock

Livelock is a form of liveness failure in which a thread, while not blocked, still cannot make progress because it keeps retrying an operation that will always fail. Livelock often occurs in transactional messaging applications, where the messaging infrastructure rolls back a transaction if a message cannot be processed successfully, and puts it back at the head

of the queue. If a bug in the message handler for a particular type of message causes it to fail, every time the message is dequeued and passed to the buggy handler, the transaction is rolled back. Since the message is now back at the head of the queue, the handler is called over and over with the same result. (This is sometimes called the *poison message* problem.) The message handling thread is not blocked, but it will never make progress either. This form of livelock often comes from overeager error-recovery code that mistakenly treats an unrecoverable error as a recoverable one.

Livelock can also occur when multiple cooperating threads change their state in response to the others in such a way that no thread can ever make progress. This is similar to what happens when two overly polite people are walking in opposite directions in a hallway: each steps out of the other's way, and now they are again in each other's way. So they both step aside again, and again, and again. . .

The solution for this variety of livelock is to introduce some randomness into the retry mechanism. For example, when two stations in an ethernet network try to send a packet on the shared carrier at the same time, the packets collide. The stations detect the collision, and each tries to send their packet again later. If they each retry *exactly* one second later, they collide over and over, and neither packet ever goes out, even if there is plenty of available bandwidth. To avoid this, we make each wait an amount of time that includes a random component. (The ethernet protocol also includes exponential backoff after repeated collisions, reducing both congestion and the risk of repeated failure with multiple colliding stations.) Retrying with random waits and backoffs can be equally effective for avoiding livelock in concurrent applications.

Summary

Liveness failures are a serious problem because there is no way to recover from them short of aborting the application. The most common form of liveness failure is lock-ordering deadlock. Avoiding lock ordering deadlock starts at design time: ensure that when threads acquire multiple locks, they do so in a consistent order. The best way to do this is by using open calls throughout your program. This greatly reduces the number of

places where multiple locks are held at once, and makes it more obvious where those places are.